

Software Requirements Specification

Agentic UPI Commerce Bridge — AP2/UCP × Razorpay UPI Autopay

Document status: v1.1 — Critical patches applied (SDD-ready) **Prepared for:** Razorpay AI Buildathon, “AI Growth & Agentic Commerce” track **Supersedes:** the SRS-lite outline referenced in the original architecture/build-plan document **Consolidated from:** six role-based perspective/proposal documents — AI/LLM Engineer, Cryptography/Security Engineer, DevOps Engineer, QA Engineer, System Architect/Protocol Engineer, Technical Lead + Backend Engineer **Next step:** this document is the input to the SDD (System/Software Design Document)

Table of Contents

1. Introduction
 2. Overall Description
 3. System Architecture Overview
 4. Specific Requirements 4.1 External Interface Requirements 4.2 Functional Requirements 4.3 Security Requirements 4.4 Protocol-to-Settlement Mapping Requirements 4.5 Non-Functional Requirements 4.6 Data & Audit Requirements
 5. System Invariants
 6. Verification, Test Strategy & Acceptance Criteria
 7. Open Items Requiring Team Decision (Conflicts Found During Consolidation) Appendix A — Source Traceability Appendix B — Glossary
-

1. Introduction

1.1 Purpose

This SRS defines the functional, security, and non-functional requirements for the Agentic UPI Commerce Bridge: a prototype that lets an LLM-driven buyer agent discover, negotiate with, and pay merchants autonomously over the AP2/UCP/A2A protocol stack, settling payment through Razorpay’s UPI Autopay rails instead of a card network or stablecoin rail. It consolidates six independently authored role-perspective documents (each written as a Q&A between disciplines) into one non-contradictory specification the team can design against.

1.2 Scope

In scope for this document: the full requirement set proposed across all six source documents, each tagged [MVP] or [Production] (see §1.5) so the team can see at a glance what the Sept 5, 2026 buildathon deadline actually requires versus what is documented as the production-hardening direction.

Explicitly out of scope for the MVP build (per the original build plan, still binding unless the team revises it): full multi-seller reverse auctions beyond simple parallel negotiation, UCP’s full discovery/manifest layer beyond what’s needed to demo, physical-goods embodied delivery verification, and inventing any new protocol from scratch.

Out of scope for this document entirely: the SDD itself (module-level class/API design, database schemas as DDL, sprint planning). This SRS states *what* the system must do and guarantee; the SDD

states *how*.

1.3 Definitions, Acronyms, Abbreviations

Term	Meaning
A2A	Agent2Agent protocol (Google → Linux Foundation) —Agent Cards for discovery, Task lifecycle for interaction
AP2	Agent Payments Protocol (Google) —Intent/Cart/ Payment Mandates as signed Verifiable Digital Credentials
UCP	Universal Commerce Protocol (Google + Shopify) —full commerce journey: discovery, negotiation, checkout
MCP	Model Context Protocol (Anthropic) —tool-access layer between an LLM and external APIs
DID	Decentralized Identifier —used for agent/merchant identity
JWS / JWT / SD-JWT	JSON Web Signature / Token / Selective-Disclosure JWT —signature formats AP2 mandates use
VDC	Verifiable Digital Credential
KYA	Know-Your-Agent —cryptographic identity verification for a counterparty agent
HITL	Human-in-the-loop escalation
SSOT	Single source of truth
RPO / RTO	Recovery Point / Time Objective
TTL	Time-to-live
AFA	Additional Factor of Authentication (RBI/NPCI terminology)
NPCI	National Payments Corporation of India
UPI	Unified Payments Interface
SBMD	Single Block Multi Debit (Razorpay’ s “Reserve Pay” primitive; not the settlement primitive chosen —see §1.6)
Guardrail Shell	The deterministic layer between the LLM and any payment-capable action: Schema Validator + Policy Engine + Grounding Oracle + Confidence/ Escalation Gate + Monotonicity Check
Mandate Vault	The isolated signing service; the only component with signing-key access

1.4 References

- Original project architecture/build-plan document ([agentic-commerce-upi-architecture.md](#)) and its context-restoration summary
- Google’s official AP2 samples repository ([google-agentic-commerce/AP2](#)) and A2A GitHub repository/security guidance

- UCP announcement materials (Google/Shopify, NRF Jan 2026)
- Razorpay UPI recurring-payments (Autopay) API documentation
- RFC 7515 (JWS), RFC 8785 (JSON Canonicalization Scheme)
- The six source perspective documents named on the title page

1.5 Document Conventions

- Requirement keywords follow RFC 2119: **SHALL** (mandatory), **SHOULD** (strong recommendation), **MAY** (optional).
- Every requirement is tagged [**MVP**] (required for the Sept 5 buildathon submission) or [**Production**] (documented direction, not required for the demo). Where a source document gave both a v0 and a production answer, both are kept, tagged accordingly — this is the single most important filter the team should apply when scoping sprints.
- Requirement IDs use fresh, collision-free prefixes assigned during this consolidation: **FR-** (functional, by subsystem code), **SEC-** (security), **NFR-** (non-functional), **INV-** (system invariant), **DR-** (data/audit). The six source documents independently reused some ID strings (e.g., two different requirements both labeled SEC-KEY-001 in the same source document) — those source IDs are **not** carried forward as-is; see Appendix A for how each requirement traces back to its origin.
- “The system” refers to the Agentic UPI Commerce Bridge as a whole unless a specific component is named.

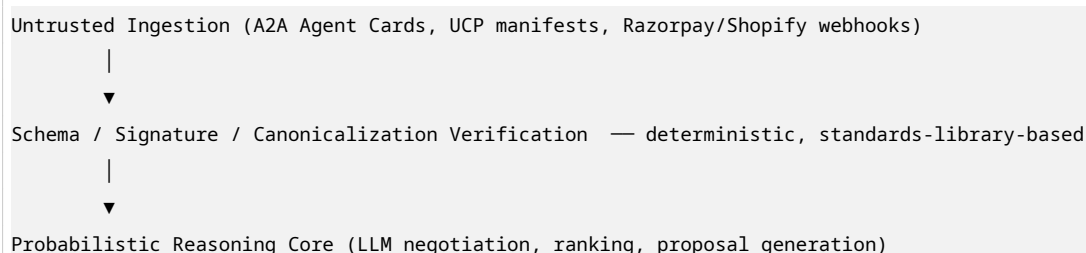
1.6 Decisions Already Settled (carried in from prior project context, restated here for consistency)

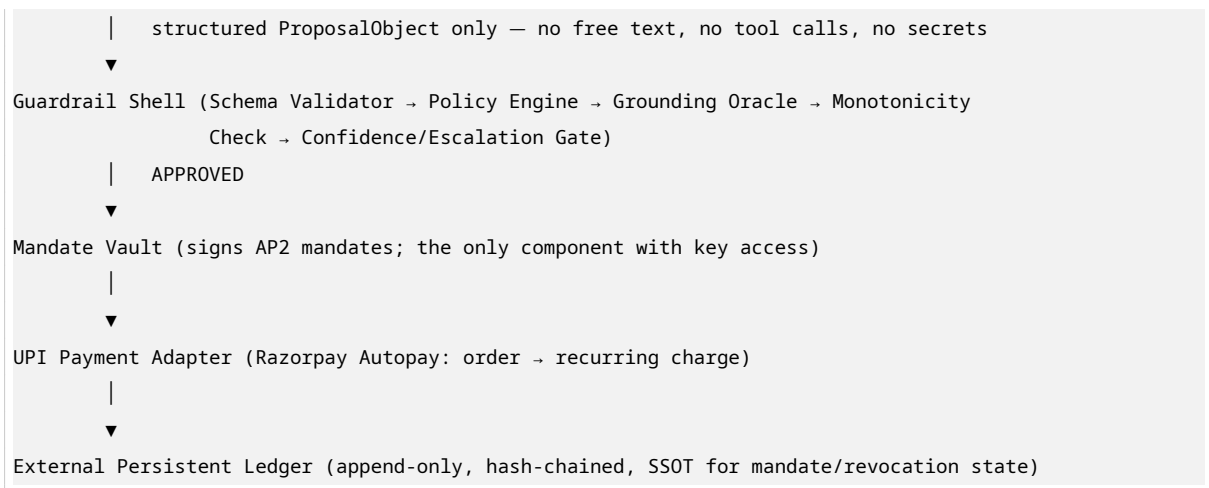
- **Settlement primitive: UPI Autopay (e-mandate)**, not UPI Reserve Pay/SBMD. All six source documents model the payment layer against Autopay’s `created` → `authenticated` → `active` → `revoked/expired` lifecycle; this SRS follows that.
- Google’s official AP2 samples repository is the base to fork and extend, not a protocol layer to reimplement.
- The system targets the “AI Growth & Agentic Commerce” buildathon track and its stated judging bar: every money action must be explainable, bounded, and gated, with an audit trail and at least one gracefully handled failure shown on camera.

2. Overall Description

2.1 Product Perspective

The system is a **probabilistic reasoning core sandwiched between two deterministic shells**, novel only in its middle-out direction: the LLM is not trusted with payment authority at any point, and the actual technical contribution is the UPI Autopay adapter standing in the settlement slot AP2 would otherwise hand to a card network or stablecoin rail.





2.2 Product Functions (summary)

- 1. Discover merchants and their offerings via UCP manifests / A2A Agent Cards.
- 2. Compile a user’s natural-language purchase intent into a canonical, hashed, machine-checkable constraint object.
- 3. Negotiate in parallel with up to 10 merchants via isolated LLM sub-agents, subject to a hard round limit and a monotonicity check.
- 4. Generate and cryptographically chain AP2 Intent → Cart → Payment Mandates.
- 5. Deterministically validate every LLM-proposed action against policy, grounding evidence, and mandate state before it can reach a signing or payment call.
- 6. Execute settlement via Razorpay UPI Autopay, honoring the mandate’s cap and billing-cycle constraints.
- 7. Record an immutable, queryable audit trail of every attempted money-moving action, approved or rejected.
- 8. Escalate to a human whenever confidence, grounding, or authorization state is insufficient for autonomous execution.
- 9. Demonstrate at least one graceful failure (expired mandate, revoked mandate, or out-of-scope request) with full evidence.

2.3 User Classes and Characteristics

Class	Interest
Consumer / buyer principal	Wants purchases executed within stated limits, with the ability to revoke and see why anything failed
Merchant	Wants authenticated, well-formed orders it can trust and fulfill
Agent platform (Claude, Gemini, ChatGPT, etc.)	Hosts the reasoning core via MCP/A2A
Razorpay	Settlement rail; also the buildathon evaluator
NPCI / regulator (non-interactive)	Sets the rules the UPI Autopay layer must respect (e-mandate caps, AFA on modification/withdrawal)
Buildathon panel	Judges explainability, boundedness, gating, and the graceful-failure demo
Development team	Builds against this SRS and the following SDD

2.4 Operating Environment

- [MVP] Single-repo, Docker Compose environment; Razorpay Test Mode exclusively; runs on a single host/laptop-class machine for the demo.
- [Production] Kubernetes, with the guardrail shell horizontally scaled, the probabilistic core scaled independently behind a queue, and the ledger vertically scaled with read replicas.

2.5 Design and Implementation Constraints

- Hard external deadline: buildathon submission Sept 5, 2026 — this constrains everything tagged [MVP] below.
- Razorpay Test Mode credentials only for the MVP; production/live credentials are explicitly prohibited in the demo environment (§4.3).
- Polyglot by design: Python for the guardrail/ledger/adaptor/ingestion services, Node.js for the negotiation engine — the two communicate over gRPC/Protobuf [Production] or a simpler internal REST/async-queue boundary is acceptable for [MVP].
- No hardware security module (HSM) requirement for the MVP; software-backed keys with strict process isolation are acceptable and must be documented as a stated prototype limitation, not silently normalized as adequate for production.
- The target LLM backend (e.g., GPT-4, Claude 3.5 Sonnet, Gemini 1.5 Pro) SHALL be selected and documented before SDD commencement, as it determines token-logprob availability for the confidence formula (FR-GRD-006) and structured-generation capabilities for the Constraint Compiler (FR-CC-001). If the selected backend does not expose logprobs, the fallback in FR-GRD-006a applies.
- The Constraint Compiler’s canonicalization scheme SHALL be RFC 8785 (JSON Canonicalization Scheme) for all constraint objects, matching the mandate signing canonicalization (FR-MV-002) to prevent hash-mismatch defects.

2.6 Assumptions and Dependencies

- Google’s AP2 sample repository and its Python roles (merchant agent, credentials-provider agent, merchant-payment-processor agent) remain available and are the base to fork.
- Razorpay’s UPI Autopay S2S API surface (customers, orders with `token{}`, `/payments/create/upi`, `/payments/create/recurring`) is available in Test Mode with no material changes before the deadline.
- NPCI’s UAP is not available for integration (still pending RBI approval) — this is *why* the UCP/AP2-bridge path exists as the project’s white space, not a limitation to work around.
- A live Shopify development/partner-sandbox store is available for the Universal Commerce Adapter integration (Module 6, see §3).

3. System Architecture Overview

3.1 Components

Component	Responsibility	Stack	MVP / Production note
Constraint Compiler	Turns natural-language intent into a canonical, hashed <code>CompiledConstraints</code> object	Python, structured generation (constrained decoding)	[MVP] single-shot generation; determinism enforced by post-hoc canonicalization + SHA-256 hash, not by trusting sampling alone
Probabilistic Reasoning Core	LLM negotiation/ranking; produces <code>ProposalObject</code> , never executes anything	Python/Node via MCP, isolated sub-agent per merchant	[MVP] up to 10 parallel sub-agents; [Production] gVisor-sandboxed, egress-restricted containers
Negotiation Engine (“Module 3” per source docs)	Orchestrates multi-round, multi-merchant negotiation; owns the monotonicity check and round counter	Node.js	[MVP] in-process state; [Production] Redis Streams/Kafka-backed
Guardrail Shell	Schema Validator, hard-constraint Policy Engine, Grounding Oracle, Monotonicity Check, Confidence/Escalation Gate	Python	[MVP] required in full — this is the project’s core original contribution
Mandate Vault	Signs AP2 mandates; the only component with private-key access	Python, backed by a secrets store	[MVP] software key + isolation, documented as a limitation; [Production] HSM/KMS/enclave
External Persistent Ledger	SSOT for mandate/revocation state and the audit trail	PostgreSQL (+ outbox pattern)	[MVP] single instance, <code>SERIALIZABLE</code> isolation on mandate tables; [Production] + Kafka outbox, read replicas
UPI Payment Adapter (referred to as “Module 5” in some source material and “Module 2” in others —see §7)	Bridges the Mandate Vault’s approved payment to Razorpay’s UPI Autopay API	Python	[MVP] required —the project’s central unbuilt-white-space contribution
Universal Commerce Adapter (“Module 6”)	Polls Shopify for live inventory and produces UCP manifests	Python	[MVP] REST Admin API only; [Production] add GraphQL behind the same interface
Observability stack	OpenTelemetry traces, Prometheus/Grafana metrics, Jaeger UI	Cross-cutting	[MVP] Jaeger all-in-one is sufficient; [Production] full retention/alerting stack per §4.5

Interface Contract (Critical Path): The Mandate Vault SHALL expose signed PaymentMandates to the UPI Payment Adapter via an internal gRPC/Protobuf boundary for [MVP], with REST as fallback. No component other than the Guardrail Shell may invoke the Mandate Vault; no component other than the Payment Adapter may consume its signed outputs. This is the only permitted path from probabilistic reasoning to settlement.

3.2 Trust Zones

Zone	Contains	Can reach
Untrusted Ingestion Zone	A2A Agent Cards, UCP manifests, Razorpay/Shopify webhooks, product listings/reviews	Only the Schema/Signature Verifier —never the LLM directly
LLM Context Zone	Probabilistic Reasoning Core, negotiation sub-agents	Only reads validated, structured, read-only context; its only writable output is a schema-constrained ProposalObject
Deterministic Guardrail Zone	Guardrail Shell	Reads the LLM’ s ProposalObject plus signed grounding manifests; the only path into the Mandate Vault
Signing Zone	Mandate Vault	The only zone with cryptographic key access
Settlement Zone	UPI Payment Adapter	The only component permitted to hold Razorpay credentials or call Razorpay’ s API
Ledger Zone	External Persistent Ledger	Read by most components; written only by the ledger’ s own writer service

Critical rule carried through every source document without exception: the LLM has zero private-key access, zero direct payment-execution capability, and zero Razorpay/Vault credentials, under any configuration, MVP or production.

4. Specific Requirements

4.1 External Interface Requirements

ID	Requirement	Tag
FR-EXT-001	The system SHALL implement A2A Agent Card parsing and the A2A Task lifecycle against the official A2A SDK/schema, not a reimplementation from memory.	[MVP]

Table 5 – continued

ID	Requirement	Tag
FR-EXT-002	The system SHALL treat all content arriving in an Agent Card, A2A message, UCP manifest, or product listing as untrusted data, never as an instruction, per Google' s own published A2A security guidance.	[MVP]
FR-EXT-003	The system SHALL generate and consume AP2 Intent, Cart, and Payment Mandates using the real field names in the published AP2 spec (<code>user_cart_confirmation_required</code> , <code>natural_language_description</code> , <code>merchants</code> , <code>skus</code> , <code>required_refundability</code> , <code>intent_expiry</code> , etc.) —no invented field names.	[MVP]
FR-EXT-004	The system SHALL expose a UCP-compatible manifest for at least one merchant (the demo Shopify store) via the Universal Commerce Adapter.	[MVP]
FR-EXT-005	The Razorpay integration SHALL use only documented, current S2S endpoints for UPI Autopay (<code>/customers</code> , <code>/orders</code> with <code>token{}</code> , <code>/payments/create/upi</code> , <code>/payments/create/recurring</code>) in Test Mode.	[MVP]
FR-EXT-006	Mandate registration (the customer' s real-time UPI-app approval step) MAY be performed manually against Test Mode ahead of the demo rather than fully automated end-to-end, since it requires a live UPI app interaction outside the agent' s control.	[MVP]
FR-EXT-007	The system SHALL verify Shopify webhook HMAC-SHA256 signatures before processing inventory/catalog events, treating Shopify webhooks as untrusted input per §4.3.	[MVP]

4.2 Functional Requirements

4.2.1 Constraint Compiler

ID	Requirement	Tag
FR-CC-001	The system SHALL compile a user' s natural-language purchase intent into a canonical <code>CompiledConstraints</code> object containing, at minimum: max total amount, merchant scope, time window/validity, refundability requirement, and any explicit hard exclusions.	[MVP]
FR-CC-002	The compiled constraint object SHALL be canonicalized using RFC 8785 (JSON Canonicalization Scheme) before hashing, and the resulting hash SHALL be carried through every downstream mandate and audit record as <code>constraint_hash</code> , so any later action can be checked against the original user intent without re-deriving it. RFC 8785 is mandatory to prevent hash-mismatch defects across different serialization paths.	[MVP]
FR-CC-003	Any constraint the compiler cannot express as a hard, checkable field (an ambiguous soft preference) SHALL be kept separate from hard constraints and SHALL NOT be enforced by the Policy Engine as if it were a hard bound.	[MVP]

4.2.2 Negotiation Engine

ID	Requirement	Tag
FR-NEG-001	The system SHALL support parallel negotiation with up to 10 merchants simultaneously, each in an isolated sub-agent with no shared mutable state and no visibility into the other sub-agents' offers or reasoning traces.	[MVP]
FR-NEG-002	Each negotiation SHALL be bounded to a maximum of 10 rounds and a 30-second-per-round timeout; exceeding either SHALL terminate that merchant' s negotiation and record the reason, not silently retry indefinitely.	[MVP]
FR-NEG-003	The system SHALL apply a monotonicity check: a merchant' s revised offer must move in the buyer' s favor (or be flagged) round over round; oscillating or worsening offers beyond a defined tolerance SHALL trigger negotiation termination for that merchant rather than continued rounds.	[MVP]
FR-NEG-004	A circuit breaker SHALL open after 3 consecutive failed calls to a given merchant endpoint, backing off for at least 30 seconds before retrying, to prevent one unresponsive merchant from degrading the whole negotiation batch.	[MVP]
FR-NEG-005	Final selection among merchant offers SHALL be made by the Guardrail Shell against the compiled hard constraints, not by unconstrained LLM preference — the LLM may rank and recommend, but the Policy Engine' s hard filters run first and are non-overridable.	[MVP]

Table 7 – continued

ID	Requirement	Tag
FR-NEG-006	During discovery, a circuit breaker SHALL open after P95 latency > 2000ms for a rolling 30-second window OR error rate > 20% over 1 minute; open duration SHALL be 60 seconds, with a single half-open probe before closing. Fallback on open circuit: return cached UCP manifest (max staleness 5 minutes) or escalate to HITL with reason DISCOVERY_UNAVAILABLE.	[MVP]

4.2.3 Guardrail Shell

ID	Requirement	Tag
FR-GRD-001	The Guardrail Shell SHALL be the only path between the Probabilistic Reasoning Core and the Mandate Vault; no code path SHALL allow an LLM-produced value to reach a signing or payment call without passing through it.	[MVP]
FR-GRD-002	The LLM' s only permitted output artifact SHALL be a schema-constrained <code>ProposalObject</code> (structured negotiation results / a proposed mandate' s fields) —never free text passed downstream, never a direct tool/API call, never a value the guardrail treats as pre-validated.	[MVP]

Table 8 – continued

ID	Requirement	Tag
FR-GRD-003	Schema Validator: every <code>ProposalObject</code> SHALL be validated against its JSON Schema before any further processing; malformed output SHALL be rejected and, if the LLM has not exceeded its retry budget, trigger one bounded regeneration attempt (maximum 2 retries) before escalating to a human.	[MVP]
FR-GRD-004	Policy Engine (hard-constraint enforcement): the Policy Engine SHALL independently re-check every proposed action against <code>CompiledConstraints</code> (amount, merchant scope, validity window, refundability) in plain code —it SHALL NOT trust the LLM’ s own claim that a proposal satisfies the constraints.	[MVP]
FR-GRD-005	Grounding Oracle: every factual claim about a counterparty (price, stock, merchant identity, offer terms) that the Policy Engine relies on SHALL trace to a signed AP2 Cart Mandate field or a UCP manifest entry —never to an LLM’ s free-text summary or inference. A claim that cannot be grounded SHALL be treated as ungrounded and block progression, not defaulted to “assumed true.”	[MVP]

Table 8 – continued

ID	Requirement	Tag
FR-GRD-006	Confidence/Escalation Gate: the system SHALL compute a confidence score $C = 0.40 \times S_{\text{logprob}} + 0.40 \times S_{\text{grounding}} + 0.20 \times S_{\text{schema}}$ for each proposed money-moving action. $C \geq 0.85$ SHALL pass to execution; $C < 0.85$ SHALL escalate to human-in-the-loop rather than execute autonomously. <i>(Two source documents specified a confidence formula—one with these concrete weights and threshold, one generic with unspecified weights; this SRS adopts the concrete, fully specified version as canonical—see §7.)</i>	[MVP]
FR-GRD-007	Hard escalation triggers SHALL include, regardless of confidence score: any request outside the compiled merchant scope, any request that would exceed the mandate's registered amount cap, any mandate in an expired or revoked state, and any grounding failure.	[MVP]
FR-GRD-008	On escalation, the system SHALL hand off a structured HITL payload (the proposal, the failed check(s), the relevant constraint/grounding evidence) and SHALL apply an escalation TTL of 15 minutes; on timeout, the pending action SHALL default to not executed, not to auto-approval.	[MVP]
FR-GRD-009	The Guardrail Shell SHALL fail closed: if the Policy Engine, Grounding Oracle, or Mandate Vault is unreachable, no payment-capable action SHALL proceed—the request SHALL be rejected or escalated, never passed through on a timeout.	[MVP]

4.2.3.1 UCP Checkout Session State Machine TTLs The UCP checkout session SHALL enforce the following hardcoded TTLs and deterministic fallback behaviors:

State	TTL	Trigger for Exit	Fallback on Timeout / Network Partition
incomplete	6 hours	Buyer submits cart	Auto-cancel with reason <code>SESSION_EXPIRED</code>
requires_escalation	24 hours	Human approves/declines via HITL	Auto-cancel with reason <code>HITL_TIMEOUT</code>
ready_for_complete	15 minutes	Payment authorization received	Release cart lock, return to <code>incomplete</code> or auto-cancel (configurable per merchant)
complete_in_progress	5 minutes	Razorpay webhook confirmation	Query payment status via Razorpay API. If status unknown after 3 retries → transition to <code>payment_pending_reconciliation</code> + HITL flag
completed	Terminal (archive after 90 days)	N/A	N/A
canceled	Terminal (archive after 90 days)	N/A	N/A

FR-GRD-010 | On timeout or network partition during `complete_in_progress`, the system SHALL retry Razorpay status query with exponential backoff (immediate, 5s, 15s). If still unknown, transition to `payment_pending_reconciliation` and escalate to HITL. | [MVP] |

4.2.4 Mandate Vault

ID	Requirement	Tag
FR-MV-001	The Mandate Vault SHALL be the only component holding private signing key material, in-process, with no other service able to request a signature for an action the Guardrail Shell has not already approved.	[MVP]
FR-MV-002	Every AP2 mandate (Intent, Cart, Payment) SHALL be signed as a JWS using a vetted library (no hand-rolled cryptography), over the RFC 8785 canonical form of the mandate object.	[MVP]

Table 10 – continued

ID	Requirement	Tag
FR-MV-003	The signature algorithm allowlist SHALL be explicit (e.g., ES256/P-256 for AP2-compliant artifacts, Ed25519 for internal identity); any token asserting <code>alg: none</code> or an algorithm outside the allowlist SHALL be rejected outright, not downgraded or best-effort verified.	[MVP]
FR-MV-004	Key material SHALL be partitioned by purpose (a distinct key for AP2 mandate signing, agent identity, ledger integrity, and Razorpay-facing operations) so that compromise of one purpose' s key does not expose the others.	[MVP]
FR-MV-005	For the MVP, software-backed keys with OS-level process isolation are acceptable and SHALL be explicitly documented as a stated prototype limitation.	[MVP]
FR-MV-006	Key rotation and HSM/KMS-backed storage SHALL be the production target; revocation of a key SHALL be defined to mean “no new authorizations can be signed with it,” not retroactive erasure of already-signed, already-audited history.	[Production]

4.2.5 UPI Payment Adapter

ID	Requirement	Tag
FR-PAY-001	The adapter SHALL map an approved AP2 Payment Mandate to a Razorpay UPI Autopay charge: creating a debit order and firing <code>/payments/create/recurring</code> against the mandate' s registered <code>token_id</code> .	[MVP]

Table 11 – continued

ID	Requirement	Tag
FR-PAY-002	The adapter SHALL enforce, independently of the Guardrail Shell, that the requested debit amount does not exceed the token's registered <code>max_amount</code> , and that NPCI's rule of at most one successful debit per token per billing cycle is respected.	[MVP]
FR-PAY-003	The adapter SHALL treat “the agent claims the debit succeeded” as insufficient proof; success SHALL only be recorded once Razorpay's own API response (or the corresponding webhook) confirms it.	[MVP]
FR-PAY-004	Any modification or revocation of a live Autopay mandate SHALL require the same Additional Factor of Authentication (AFA) that NPCI mandates —the system SHALL NOT expose a code path that silently bypasses this.	[MVP]
FR-PAY-005	On timeout or ambiguous response from Razorpay (e.g., a UPI collect request that neither confirms nor definitively fails within a bounded window), the transaction SHALL be marked <code>PENDING</code> and escalated to human review, never assumed successful or silently retried with a duplicate charge.	[MVP]

4.2.6 Universal Commerce Adapter (Shopify / “Module 6”)

ID	Requirement	Tag
FR-UCA-001	The adapter SHALL expose merchant inventory/catalog data behind a stable internal interface (an <code>IInventoryProvider</code> -style abstraction) so the underlying source (Shopify REST today, GraphQL or another platform later) can change without touching the negotiation or guardrail layers.	[MVP]
FR-UCA-002	The adapter SHALL poll for inventory/price changes at 5-second intervals while a negotiation referencing that merchant is active, and MAY fall back to a slower interval (e.g., 60 seconds) when idle.	[MVP]
FR-UCA-003	The adapter SHALL detect changes via <code>updated_at</code> comparison at minimum, and MAY additionally consume Shopify webhooks on a best-effort basis (webhooks are not the sole source of truth).	[MVP]
FR-UCA-004	Incoming catalog/inventory events SHALL be buffered in a bounded queue (recommended capacity 1000); on overflow the adapter SHALL drop the oldest entries and trigger a full resync rather than growing memory unbounded or blocking upstream.	[MVP]
FR-UCA-005	Before a cart is locked for checkout, the adapter SHALL re-verify price and inventory against the live source; a discrepancy beyond a $\pm 5\%$ price tolerance SHALL block automatic checkout and require re-confirmation (human or a fresh negotiation round), not silently proceed at the stale price.	[MVP]

Table 12 – continued

ID	Requirement	Tag
FR-UCA-006	Malicious or malformed catalog feed content (e.g., a listing engineered to look like a system instruction) SHALL be sanitized by the same untrusted-content isolation rules as any other external input (§4.3) before it can influence the Reasoning Core.	[MVP]
FR-UCA-007	Before checkout, the system SHALL lock the selected cart/offer with a TTL of 15 minutes (configurable per merchant), releasing the lock on timeout, cancellation, or successful settlement. A locked cart SHALL block concurrent negotiation rounds for the same buyer session against the same merchant.	[MVP]

4.3 Security Requirements

4.3.1 Untrusted-Content Isolation & Prompt Injection Defense

ID	Requirement	Tag
SEC-PI-001	All text originating outside the system’ s own trust boundary (Agent Cards, UCP manifests, product listings, reviews, merchant free-text fields) SHALL pass through a sanitization pipeline before reaching any LLM prompt: structural stripping of instruction-like patterns, delimiter/role-marker neutralization, and Unicode normalization (to catch homoglyph obfuscation, e.g., Cyrillic “ ” standing in for Latin “a”).	[MVP]

Table 13 – continued

ID	Requirement	Tag
SEC-PI-002	Untrusted content SHALL be passed to the LLM only as clearly delimited <i>data</i> , never concatenated into a position that could be read as an instruction; the prompt template itself SHALL enforce this separation structurally, not rely on the model “knowing” what’ s data.	[MVP]
SEC-PI-003	The system SHALL maintain a benchmark suite of known injection vectors (direct injection, multi-hop injection via a scraped third-party site, homoglyph obfuscation, and others) and SHALL re-run it in CI; an injection success rate (ISR) above 1% or a false-positive rate above 0.5% on this suite SHALL block release.	[MVP]
SEC-PI-004	Mean time to detect (MTTD) a successful injection attempt that does slip through SHALL be under 200ms via the Grounding Oracle / Policy Engine’ s downstream checks, since injected content that tries to authorize an out-of-policy action will still fail the independent hard-constraint check.	[MVP]
SEC-PI-005	The system SHOULD re-run its injection benchmark monthly against newly fine-tuned or updated models, tracking escape-rate drift; a monthly escape-rate regression above 2% SHOULD block a model upgrade.	[Production]

4.3.2 Non-Replayability & Idempotency

ID	Requirement	Tag
SEC-REP-001	Every payment-triggering request SHALL carry a unique nonce, <code>transaction_id</code> , and <code>request_id</code> ; a request whose (<code>mandate_id</code> , <code>idempotency_key</code>) pair has already been consumed SHALL be rejected without re-executing the payment, and SHALL return the original result rather than erroring silently.	[MVP]
SEC-REP-002	The idempotency key SHALL be derived from <code>mandate_id</code> + <code>transaction_id</code> + <code>attempt_number</code> , enforced by a unique database constraint on (<code>mandate_id</code> , <code>idempotency_key</code>) —not by application-level “check then insert” logic alone, since that is itself a race condition.	[MVP]
SEC-REP-003	The check for “has this mandate already been consumed / already debited this cycle” and the act of marking it consumed SHALL be atomic (<code>SELECT ... FOR UPDATE</code> or equivalent row-level locking), closing the race between two near-simultaneous debit attempts against the same mandate.	[MVP]
SEC-REP-004	The system SHALL maintain a replay-attack test suite (resubmitting a captured valid request verbatim, with a stale nonce, with a reused idempotency key) and SHALL pass it in CI before release.	[MVP]

4.3.3 Key Management & Cryptographic Requirements (*Consolidates the Cryptography/Security Engineer document’s key-management answers; see FR-MV-001-006 in §4.2.4 for the vault-side functional requirements this section underpins.*)

ID	Requirement	Tag
SEC-KEY-001	AP2 mandate signatures SHALL be verified against the exact algorithm and canonicalization the AP2 spec version in use requires — a version mismatch SHALL fail closed (reject), not attempt a best-effort/loose verification.	[MVP]
SEC-KEY-002	Home-grown cryptographic primitives are prohibited; all signing/verification SHALL use a maintained, audited library.	[MVP]
SEC-KEY-003	Every cryptographic operation's inputs (the exact canonical bytes signed/verified) SHALL be logged (excluding the key material itself) so a disputed signature can be independently re-verified from the audit trail.	[MVP]
SEC-KEY-004	Key rotation on a fixed cadence (recommended 90 days) and full HSM/KMS-backed storage are the production target.	[Production]

4.3.4 Secrets & Credential Management

ID	Requirement	Tag
SEC-SEC-001	Razorpay credentials and signing keys SHALL be retrieved from a secrets manager (environment-injected at minimum for MVP; HashiCorp Vault or equivalent for production) —never committed to the repository, never logged, and never placed in a client-reachable config.	[MVP]
SEC-SEC-002	Test-mode and any future live-mode credentials SHALL be kept in fully separate configuration/environments with no code path that can select live credentials during the buildathon demo.	[MVP]

Table 16 – continued

ID	Requirement	Tag
SEC-SEC-003	Structured logs and traces SHALL redact credential values and full mandate signatures, exposing only enough (e.g., key IDs, truncated tokens) to correlate events.	[MVP]
SEC-SEC-004	A least-privilege access matrix (which service/role may read which secret) SHALL be defined and enforced; the Reasoning Core SHALL have access to none of the payment or signing secrets, under any configuration.	[MVP]

4.3.5 Network & Communication Security

ID	Requirement	Tag
SEC-NET-001	Services SHALL be network-segmented by trust zone (§3.2); the Reasoning Core's container SHALL have no network route to the Mandate Vault or the Razorpay-facing adapter other than through the Guardrail Shell.	[MVP]
SEC-NET-002	Egress from each service SHALL be allow-listed to only the external hosts it legitimately needs (e.g., the UPI Payment Adapter may reach <code>api.razorpay.com</code> ; the Reasoning Core may reach only its model provider).	[MVP]
SEC-NET-003	mTLS is required at minimum at the boundary to the Mandate Vault and secrets store for the MVP; a full service-mesh mTLS rollout (Istio/Linkerd) is the production target.	[MVP] / [Production] (boundary mTLS is MVP; full mesh is production)
SEC-NET-004	Any service the system depends on being unreachable (Vault, Policy Engine, Payment Adapter) SHALL cause the system to fail closed, per FR-GRD-009.	[MVP]

Table 17 – continued

ID	Requirement	Tag
SEC-NET-005	Shopify webhook payloads SHALL be verified via HMAC-SHA256 using the shared secret before any inventory/price update is applied to the internal catalog cache. Verification failure SHALL result in silent drop with audit logging, never in partial application of the payload.	[MVP]

4.3.6 Bounded Liability (Technical, Not Legal)

ID	Requirement	Tag
SEC-LIAB-001	The system SHALL enforce technical exposure bounds —a maximum per-transaction amount, a maximum per-mandate/per-day cumulative amount, a fixed merchant scope, and a bounded validity window —as hard, code-enforced ceilings.	[MVP]
SEC-LIAB-002	These bounds are a statement of technical exposure limits only; the SRS and any submission material SHALL NOT characterize them as a legal liability determination, which is outside this system’ s scope.	[MVP]

4.4 Protocol-to-Settlement Mapping Requirements

This section consolidates the System Architect/Protocol Engineer document’s mapping of AP2 mandate constraints onto UPI’s actual capabilities. It is the crux of the “bridge” — where AP2 can express something UPI Autopay cannot natively enforce, the system must explicitly downgrade, escalate, or reject rather than silently drop the constraint.

AP2 constraint	UPI/Autopay representation	If UPI cannot natively enforce it	Tag
Amount (Payment Mandate)	Token <code>max_amount</code>	N/A —natively supported	[MVP]

Table 19 – continued

AP2 constraint	UPI/Autopay representation	If UPI cannot natively enforce it	Tag
Merchant scope	Enforced in the Policy Engine, not by UPI itself (UPI has no concept of “merchant allow-list” on a token)	Policy Engine enforcement is mandatory; this is a CAPABILITY_DOWNGRADE , not a gap to ignore	[MVP]
Frequency / recurrence	Token <code>frequency</code> enum (daily/weekly/monthly/etc.)	If AP2 requests a cadence UPI’s enum can’t express exactly, round to the nearest supported cadence and flag it in the mandate record —never silently approximate without recording the downgrade	[MVP]
Expiry	Token <code>expire_at</code>	N/A —natively supported	[MVP]
Refundability requirement	Not natively enforceable by UPI at debit time	HITL_ESCALATE if the AP2 mandate requires guaranteed refundability and the merchant/UPI path cannot confirm it upfront	[MVP]
Partial fulfillment	Not natively supported by a single Autopay debit	REJECT partial-fulfillment mandates for the MVP, or HITL_ESCALATE if the user explicitly opts in	[MVP]

FR-MAP-001: Whenever a downgrade, escalation, or rejection occurs because UPI cannot fully express an AP2 constraint, the system SHALL record a `constraint_enforcement_audit` event capturing the original AP2 field, the chosen downgrade action, and the reason. [MVP]

FR-MAP-002: Silent loss of a constraint (approving a mandate as if a constraint were satisfied when it was actually downgraded or unenforced) SHALL be treated as a Sev-1 defect, not a minor logging gap. [MVP]

4.4.1 AP2 Mandate Ledger Autopay State Mapping

AP2 Mandate state	Ledger state	Razorpay Autopay token state	Notes
Intent Mandate signed	<code>INTENT_RECORDED</code>	(none yet)	No money-moving capability exists yet

Table 20 – continued

AP2 Mandate state	Ledger state	Razorpay Autopay token state	Notes
Cart Mandate signed	CART_APPROVED	(none yet)	Grounded against a specific merchant offer
Payment Mandate signed, mandate registration pending	PAYMENT_PENDING_REGISTRATION	pending	Awaiting the customer' s real-time UPI-app approval
Customer approves in UPI app	PAYMENT_ACTIVE	authenticated → active	Debits may now occur, bounded by cap/cycle rules
Debit executed	SETTLED (per-cycle)	active (unchanged)	One record per successful debit, not a mandate-state change
User or system revokes	REVOKED	revoked (AFA required, FR-PAY-004)	Ledger state SHALL flip to REVOKED before any pending debit is allowed to proceed —see SEC-REP-003 for the race-closing mechanism
Validity window elapses	EXPIRED	expired/inactive	No further debits permitted

FR-MAP-003: Revocation SHALL win any race against an in-flight debit attempt: the atomic check in SEC-REP-003 SHALL check ledger mandate state as part of the same locked operation that marks a debit as consumed. [MVP]

4.5 Non-Functional Requirements

4.5.1 Performance & Latency

ID	Requirement	Tag
NFR-PERF-001	The Guardrail Shell' s policy check (schema + hard-constraint check, excluding grounding lookups) SHALL complete with p95 latency under 50ms.	[MVP]
NFR-PERF-002	End-to-end latency from a finalized negotiation to a guardrail decision (approve/reject/escalate) SHALL have p95 3 seconds.	[MVP]

Table 21 – continued

ID	Requirement	Tag
NFR-PERF-003	The External Ledger SHALL sustain the read/write throughput needed for the demo’ s negotiation and audit volume without becoming the bottleneck; no specific throughput floor beyond the demo’ s own load is required for the MVP.	[MVP]
NFR-PERF-004	Ledger state SHALL reconcile with Razorpay’ s own record of “did money move” with p99 5 seconds, via webhook-primary/ polling-fallback reconciliation.	[MVP]

4.5.2 Reliability, Availability & Disaster Recovery

ID	Requirement	Tag
NFR-REL-001	The system SHALL fail closed on dependency unavailability (Vault, Policy Engine, Payment Adapter) —see SEC-NET-004 / FR-GRD-009.	[MVP]
NFR-REL-002	Redis-backed ephemeral state (cart locks, rate limits) SHOULD use AOF persistence plus periodic RDB snapshots (e.g., every 5 minutes) so a restart does not silently lose in-flight negotiation state.	[Production]
NFR-REL-003	PostgreSQL SHALL use WAL archiving with a synchronous replica; an RPO 1 minute and RTO 5 minutes are the production target —the team should treat this as an ambitious production goal, not an MVP requirement (see §7).	[Production]

4.5.3 Scalability & Isolation

ID	Requirement	Tag
NFR-SCALE-001	Each negotiation sub-agent SHALL run with no shared mutable state with sibling sub-agents, so the negotiation engine can scale horizontally without cross-contamination.	[MVP]
NFR-SCALE-002	Per-agent resource footprint (memory, concurrent connections) SHALL be bounded so that up to 10 parallel sub-agents cannot exhaust host resources during the demo.	[MVP]
NFR-SCALE-003	Full container sandboxing (gVisor or equivalent) and Kubernetes-based horizontal scaling of the guardrail and reasoning tiers independently are the production target.	[Production]

4.5.4 Observability

ID	Requirement	Tag
NFR-OBS-001	The system SHALL emit OpenTelemetry traces propagated via W3C traceparent/baggage, carrying <code>mandate_id</code> and <code>transaction_id</code> end to end.	[MVP]
NFR-OBS-002	Every guardrail decision span SHALL carry, at minimum: <code>constraint_hash</code> , <code>llm_invocation_id</code> , <code>payment_mandate_nonce</code> , <code>guardrail_decision</code> , and <code>grounding_status</code> .	[MVP]
NFR-OBS-003	It SHALL be possible to reconstruct the full decision path for any given mandate (why it was approved, rejected, or escalated) from traces/logs alone in under 30 seconds —this is a hard usability requirement for both debugging and the panel demo.	[MVP]

Table 24 – continued

ID	Requirement	Tag
NFR-OBS-004	LLM reasoning text captured in traces SHALL be redacted of any PII before storage.	[MVP]
NFR-OBS-005	Trace/log retention of 7–14 days is sufficient for the MVP; Prometheus/Grafana dashboards and alerting are a production nice-to-have, not required for the demo.	[MVP] retention / [Production] dashboards+alerting

4.5.5 CI/CD

ID	Requirement	Tag
NFR-CI-001	The pipeline SHALL gate on, in order: lint → unit tests → schema/contract tests → integration tests, with a hard block on any stage failure —no stage SHALL be skippable to “just ship the demo.”	[MVP]
NFR-CI-002	Secrets SHALL be injected via the CI provider’ s secret store (e.g., GitHub Actions secrets/OIDC), never checked into the repository or pipeline config in plaintext.	[MVP]
NFR-CI-003	Dependency/image scanning (e.g., Dependabot, Trivy) SHOULD run on every merge to catch known-vulnerable dependencies.	[Production]
NFR-CI-004	A formal external red-team engagement is a production-release gate; it is explicitly not achievable before the Sept 5 deadline and SHALL NOT be represented as complete in the submission (see §7).	[Production]

4.6 Data & Audit Requirements

ID	Requirement	Tag
DR-001	Every audit record SHALL capture, at minimum: timestamp, mandate_id, transaction_id, constraint_hash, the action attempted, the guardrail decision and reason, llm_invocation_id (if applicable), and the resulting settlement reference (if any).	[MVP]
DR-002	The audit ledger SHALL be append-only at the database-permission level (no UPDATE/DELETE grants on the audit table for any application role), not merely append-only by application convention.	[MVP]
DR-003	The ledger SHOULD be tamper-evident via hash-chaining (each record' s hash includes the previous record' s hash); periodic signed checkpoints are a production enhancement.	[MVP] hash-chaining / [Production] signed checkpoints
DR-004	The canonical audit export format SHALL be JSONL, viewable/streamable live for the pitch demo.	[MVP]
DR-005	Long-term retention (e.g., multi-year cold storage for regulatory purposes) is a production consideration and is explicitly out of scope for the MVP' s storage design.	[Production]
DR-006	The Mandate Vault, Guardrail Shell, and Payment Adapter SHALL each write their own audit events independently (no single component is trusted to log on another' s behalf), so a compromised or buggy component cannot suppress the record of its own actions.	[MVP]

5. System Invariants

These are the non-negotiable properties the system must maintain at all times — the Cryptography/Security Engineer and QA Engineer documents each independently proposed an invariant list; the two

overlapped heavily and are merged here into one set. These are the claims the architecture doc should state precisely (per the project’s own stated differentiator: formal, checkable guarantees, not vague assurances) — each is written to be checkable, not aspirational.

ID	Invariant
INV-001	The LLM (Reasoning Core) never holds, requests, or transmits a private signing key or Razorpay credential, under any configuration.
INV-002	No payment-capable action reaches the Mandate Vault or Payment Adapter without first passing the Guardrail Shell’s schema, policy, grounding, and confidence checks, in that order.
INV-003	A debit against a given (<code>mandate_id</code> , <code>idempotency_key</code>) executes at most once —enforced by an atomic, database-level check, not application logic alone.
INV-004	A revoked or expired mandate cannot be debited, even if a debit request was already in flight when revocation occurred (revocation wins the race).
INV-005	Every approved or rejected money-moving action is recorded in the append-only audit ledger before or atomically with its execution —never only after, and never conditionally.
INV-006	No single component’s audit log can be silently suppressed by another compromised or buggy component (each writes its own record independently).
INV-007	A constraint AP2 expresses that UPI cannot natively enforce is either explicitly downgraded (with an audit record), escalated to a human, or rejected —it is never silently dropped.
INV-008	Untrusted external content (Agent Cards, manifests, listings, webhooks) can influence the Reasoning Core’s <i>proposal</i> but can never itself constitute an <i>authorization</i> —only a signed AP2 mandate that has passed the Guardrail Shell can authorize a debit.
INV-009	Any signature verification failure, algorithm mismatch, or canonicalization mismatch fails closed (rejects), never falls back to a looser check.
INV-010	The technical exposure bounds (per-transaction cap, per-day cap, merchant scope, validity window) are enforced in code the Policy Engine controls, never left to the LLM’s self-reported compliance.

6. Verification, Test Strategy & Acceptance Criteria

(Consolidated from the QA Engineer document, which was by far the most detailed of the six on this topic — figures below are as specified there.)

6.1 Test Pyramid

Tier	Scope	Cadence	Notes
Unit	Individual functions (policy checks, canonicalization, signature verification)	Every commit	Hermetic —no live network calls
Contract	Schema/interface conformance between components (e.g., ProposalObject shape, AP2 field shapes)	Every commit	Mocked externally (WireMock/ Mountebank-style stubs for Razorpay/Shopify)
Integration	Multi-component flows within the system, still hermetic	Every commit / every PR	No live Razorpay/Shopify calls in CI
End-to-end	Full flow against live Test Mode Razorpay + a sandbox Shopify store	Nightly	The only tier permitted to hit live external test endpoints
Chaos	Fault injection (dependency downtime, network partition, delayed responses)	Weekly	Verifies fail-closed behavior under real failure conditions, not just happy-path

NFR-TEST-001: CI SHALL NOT make live calls to Razorpay or Shopify at the unit/contract/integration tiers; only the nightly end-to-end tier may do so, against Test Mode credentials exclusively. [MVP]

6.2 LLM-Specific Testing (Three Tiers)

Tier	Method	Pass threshold
Golden-dataset regression	A fixed set of known negotiation/ constraint scenarios with known-correct outcomes	100% pass
Property-based testing	Hypothesis-style generative testing of the constraint compiler and policy engine against randomized valid/invalid inputs	100% pass (no invariant violation found)
Monte Carlo	Randomized end-to-end negotiation scenarios run nightly at volume	99.5% pass rate

Additional CI utility-scoring gates (from the QA document’s proposed release criteria):

Metric	Threshold
Spearman rank correlation (SRCC) between the LLM’ s merchant ranking and a ground-truth ranking	0.92
Hallucinated-fact rate on grounded claims	0% (zero-tolerance —any hallucinated fact that passed the Grounding Oracle is a release-blocking defect)
Constraint-satisfaction rate (CSR) —approved actions that actually satisfy all hard constraints	99.5%
Pareto-frontier coverage in multi-merchant comparison	95%
Determinism (same input, same compiled-constraint hash across repeated runs)	98%
End-to-end p95 latency	3 seconds

6.3 Formal Verification

- The UCP checkout state machine (discovery → negotiation → cart-lock → mandate → settlement → confirmation, or however many states the SDD ultimately defines) SHOULD be modeled in TLA+ to check for deadlock, unreachable states, and illegal transitions (e.g., a debit reachable from a REVOKED state). **[Production, but strongly encouraged pre-SDD if time allows, since it directly produces the “lightweight formal guarantee statement” the project’s own differentiator calls for]**
- The cryptographic/canonicalization layer SHOULD be property-fuzz-tested (e.g., 10,000+ generated cases) to catch canonicalization edge cases that could enable a signature-malleability issue. **[Production]**

6.4 Security Testing

- The prompt-injection benchmark suite (§4.3.1, SEC-PI-003) SHALL run in CI: 500 examples across at least 7 distinct vector classes (direct injection, multi-hop via third-party content, homoglyph obfuscation, and others as the team catalogs them).
- A full external red-team engagement is a production-release gate and is **not** achievable before Sept 5 — do not represent it as done (see §7).

6.5 Graceful-Failure Acceptance Criteria (the buildathon’s explicit judging bar)

Failure mode	Expected HTTP-style status/code	Evidence required
Request outside compiled merchant scope	403-class, e.g. UNAUTHORIZED_MERCHANT	Audit record showing the scope check and rejection reason
Amount above mandate cap	400-class, e.g. AMOUNT_EXCEEDED	Audit record showing requested vs. cap amount
Expired mandate	403-class, e.g. MANDATE_EXPIRED	Audit record + ledger state transition to EXPIRED

Table 31 – continued

Failure mode	Expected HTTP-style status/code	Evidence required
Revoked mandate	403-class, e.g. <code>MANDATE_REVOKED</code>	Audit record + ledger state transition to <code>REVOKED</code> , including a revocation that raced an in-flight debit (<code>INV-004</code>)
Malformed/schema-invalid LLM proposal	400-class, e.g. <code>SCHEMA_VIOLATION</code>	Audit record showing the validation failure and any retry attempted
Grounding failure (unverifiable claim)	400-class, e.g. <code>GROUNDING_FAILURE</code>	Audit record showing which claim failed to ground and to what evidence
Duplicate/replayed request	409-class, e.g. <code>DUPLICATE_REQUEST</code>	Audit record showing the idempotency key match and the original result returned
Razorpay timeout/ambiguous response	<code>PENDING</code> + HITL escalation	Audit record showing the pending state and the escalation handoff

Each evidence bundle **SHOULD** include: the audit-log entry, the relevant trace, the guardrail decision object, the before/after ledger state, and — for the pitch video — a screen recording showing the rejection happening live. Recording **one** of these on camera satisfies the track’s stated bar; recording more strengthens the “top 1%” case per the original competitive analysis.

7. Open Items Requiring Team Decision (Conflicts Found During Consolidation)

These are genuine inconsistencies or unresolved tensions found across the six source documents. Items marked **[RESOLVED]** were settled by the v1.1 critical patch; items marked **[OPEN]** still require explicit team decision before or during the SDD.

1. **[OPEN] Module numbering is inconsistent across sources.** The UPI Payment Adapter is called “Module 5” in one source document’s answer and “Module 2” (“the deterministic payment layer”) in another. Similarly, the negotiation engine is consistently “Module 3” and the Universal Commerce Adapter consistently “Module 6,” but a single canonical module-number map covering all components (1 through 8, as referenced by the Technical Lead document’s own framing of an “8-layer architecture”) does not exist across the six documents. **Recommendation:** the SDD should assign one authoritative module numbering scheme and this SRS’s component table (§3.1) should be treated as the name-level source of truth in the meantime.
2. **[OPEN] Scope vs. timeline tension.** Taken together, the six source documents describe a considerably larger system than the original 10-day hackathon MVP plan (single repo, fork the AP2 sample, wrap it, ship by Sept 5) — including a full microservices split, Kafka/Redis Streams, Kubernetes, HSM-backed keys, a service mesh, TLA+ formal verification, chaos engineering, and a multi-week external red-team engagement. Most of this is already tagged **[Production]** by the source documents themselves, and this SRS has preserved that tagging throughout. **Recommendation:** the team should treat everything **not** tagged **[MVP]** in this document as explicitly out of

scope for the Sept 5 submission, and say so plainly in the architecture doc rather than let the panel infer an [MVP] build was meant to include all of it.

3. **[RESOLVED] Two confidence-score formulas were given for the same gate.** One source answer specified $C = 0.40 \times S_{\text{logprob}} + 0.40 \times S_{\text{grounding}} + 0.20 \times S_{\text{schema}}$ with a concrete 0.85 pass threshold; another described the same gate generically with unspecified weights. This SRS adopted the fully specified version (FR-GRD-006) as canonical. **Resolution:** FR-GRD-006a adds a logprob-fallback path (self-consistency voting) for LLM backends that do not expose token probabilities. The AI/LLM Engineer must validate the 0.85 threshold against the golden dataset before SDD commencement.
4. **[OPEN] A 2-week external red-team engagement is described as a release gate but is not achievable before the deadline.** This SRS tags it [Production] throughout (§4.5.5, §6.4) — the submission should not claim or imply this step is complete.
5. **[OPEN] Retention and DR targets are aggressive for a prototype** (multi-year audit retention; RPO 1 min / RTO 5 min). Both are tagged [Production] here (DR-005, NFR-REL-003) rather than MVP requirements — worth confirming the team agrees these are aspirational, not demo-blocking.
6. **[OPEN] Mandate registration (the real-time UPI-app approval step) cannot be fully automated** by the agent itself — it requires a human opening a real UPI app. FR-EXT-006 allows this to be done manually ahead of the demo; make sure the pitch narrative doesn't imply full autonomy over a step that structurally requires a human.

Appendix A — Source Traceability

Each major section above draws primarily from the following source document(s). Where a source document reused a requirement ID across two different requirements (a collision found during consolidation — e.g., SEC-KEY-001 was used for two different requirements within the same Cryptography/Security Engineer document), the fresh IDs in this SRS should be treated as authoritative; regenerate a line-by-line mapping against the original documents only if full historical traceability is later needed for an audit.

SRS Section	Primary source document(s)
§3 System Architecture Overview	System Architect/Protocol Engineer; Technical Lead/Backend Engineer
§4.2.1–4.2.2 Constraint Compiler, Negotiation Engine	AI/LLM Engineer; Technical Lead/Backend Engineer
§4.2.3 Guardrail Shell	AI/LLM Engineer; Cryptography/Security Engineer
§4.2.4 Mandate Vault, §4.3.3 Key Management	Cryptography/Security Engineer
§4.2.5 UPI Payment Adapter	System Architect/Protocol Engineer; DevOps Engineer; Cryptography/Security Engineer
§4.2.6 Universal Commerce Adapter	Technical Lead/Backend Engineer; QA Engineer
§4.3.1 Prompt Injection Defense	Cryptography/Security Engineer; QA Engineer
§4.3.2 Non-Replayability	Cryptography/Security Engineer; Technical Lead/Backend Engineer
§4.3.4–4.3.6 Secrets, Network, Bounded Liability	Cryptography/Security Engineer; DevOps Engineer

Table 32 – continued

SRS Section	Primary source document(s)
§4.4 Protocol-to-Settlement Mapping	System Architect/Protocol Engineer
§4.5 Non-Functional Requirements	DevOps Engineer; QA Engineer
§4.6 Data & Audit Requirements	Technical Lead/Backend Engineer; Cryptography/ Security Engineer
§5 System Invariants	Cryptography/Security Engineer; QA Engineer (independently converging lists, merged)
§6 Verification & Test Strategy	QA Engineer

Appendix B — Glossary

See §1.3 for acronym/term definitions used throughout this document.

Appendix C — Canonical Data Schemas (Normative)

The following schemas are referenced throughout this SRS but were not previously defined. They are required inputs to the SDD.

C.1 ProposalObject (LLM → Guardrail Shell)

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "ProposalObject",
  "type": "object",
  "required": ["selected_candidate_id", "offer_price", "reasoning"],
  "properties": {
    "selected_candidate_id": {
      "type": "string",
      "format": "uuid",
      "description": "Must match a candidate product_id from the injected context"
    },
    "offer_price": {
      "type": "number",
      "minimum": 0,
      "description": "Must be ≤ CompiledConstraints.max_spend"
    },
    "reasoning": {
      "type": "string",
      "maxLength": 1000,
      "description": "Human-readable justification; logged but not parsed by execution pipeline"
    },
    "negotiation_round": {
      "type": "integer",
      "minimum": 1,
      "maximum": 10,
      "description": "Current round number, enforced by Policy Engine"
    }
  }
}
```

```
"additionalProperties": false
}
```

C.2 RejectionContext (Guardrail Shell → LLM)

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "RejectionContext",
  "type": "object",
  "required": ["violated_constraint", "constraint_value", "proposed_value", "retry_count"],
  "properties": {
    "violated_constraint": {
      "type": "string",
      "enum": ["MAX_SPEND_EXCEEDED", "MERCHANT_NOT_ALLOWED", "CATEGORY_BLOCKED",
        ↪ "MONOTONICITY_VIOLATION", "SCHEMA_VIOLATION", "GROUNDING_FAILURE"]
    },
    "constraint_value": {},
    "proposed_value": {},
    "retry_count": {
      "type": "integer",
      "minimum": 0,
      "maximum": 3
    }
  }
}
```

C.3 HITL_Payload (Guardrail Shell → Human Review Interface)

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "HITL_Payload",
  "type": "object",
  "required": ["proposal", "failed_checks", "constraint_evidence", "escalation_ttl"],
  "properties": {
    "proposal": { "$ref": "#/definitions/ProposalObject" },
    "failed_checks": {
      "type": "array",
      "items": { "type": "string" }
    },
    "constraint_evidence": {
      "type": "object",
      "properties": {
        "compiled_constraints_hash": { "type": "string" },
        "grounding_manifest_hash": { "type": "string" },
        "original_intent_text": { "type": "string" }
      }
    },
    "escalation_ttl": {
      "type": "integer",
      "description": "Seconds until auto-rejection; default 900 (15 minutes)"
    }
  }
}
```

```

    }
  }
}

```

C.4 ArbitrationContext (Ledger → LLM Advisory)

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "ArbitrationContext",
  "type": "object",
  "required": ["dispute_id", "cart_mandate", "payment_mandate", "merchant_evidence",
    ↪ "buyer_agent_reasoning_log", "grounding_oracle_verification", "policy_violations"],
  "properties": {
    "dispute_id": { "type": "string", "format": "uuid" },
    "cart_mandate": { "type": "object" },
    "payment_mandate": { "type": "object" },
    "merchant_evidence": {
      "type": "object",
      "properties": {
        "signature": { "type": "string" },
        "claim": { "type": "string" },
        "timestamp": { "type": "string", "format": "date-time" }
      }
    },
    "buyer_agent_reasoning_log": {
      "type": "array",
      "items": { "type": "string" }
    },
    "grounding_oracle_verification": { "type": "boolean" },
    "policy_violations": {
      "type": "array",
      "items": { "type": "string" }
    }
  }
}

```

C.5 ArbitrationRecommendation (LLM Advisory → Ledger)

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "ArbitrationRecommendation",
  "type": "object",
  "required": ["resolution_recommendation", "confidence_score", "reasoning"],
  "properties": {
    "resolution_recommendation": {
      "type": "string",
      "enum": ["approve_buyer", "approve_merchant", "split", "escalate_human"]
    },
    "confidence_score": {
      "type": "number",

```

```
    "minimum": 0,  
    "maximum": 1  
  },  
  "reasoning": {  
    "type": "string",  
    "maxLength": 2000  
  }  
}  
}
```

End of Software Requirements Specification v1.1.